Path inversion: for every type A and every x, y there is a function from x = y to y = x carrying refl to refl

The Univalent Foundations Program

hott-book:basics.tex:local-snapshot-20261014; CC BY-SA 3.0 Unported

Editorial note. The counted statement is the author lemma labelled lem:opp (source lines 219-230); the proof span 241-254 is the author's first proof environment following it, retained line for line.

Editorial note. The author gives a second proof of the same lemma at source lines 265-271; it is not counted and not retained on the card face.

Editorial note. The retained context span 174-194 is the author's own recap of the identity type induction principle and is environment balanced.

Editorial scope. One original result of the HoTT book chapter cha:basics, counted once: symmetry of identity types (path inversion) with its definitional conversion behaviour. Only the counted lemma and its first author proof are retained; the author's second proof of the same lemma and the surrounding commentary remain uncounted in the source.

Allowed standard prerequisites (editorial).

- The identity type of a type A: for x, y in A a type x = y whose element is called a path or identification; the reflexivity element refl x in x = x.
- The induction principle for identity types (path induction): given a type family D over x, y, p and an element d(a) of D(a, a, refl a) for each a, there is an element J(D, d, x, y, p) of D(x, y, p) for every path p, satisfying the conversion rule J(D, d, a, a, refl a) equals d(a) by definition (the retained context span states exactly this rule).
- Type families and dependent functions: a family D of types over the paths of A, and the dependent product type of sections of such a family.

Source reference. Original section 1.12: sources/hott-basics/book-labelnumbers-first-edition.tex, source lines 1669–1669. The section introducing the identity type and its induction principle, cited by the retained context span at its first line. Navigation pointer only; the retained context restates the induction principle and its conversion rule in full.

Recall from original section 1.12 that this says that if

- for every $x, y : A$ and every $p : x =_A y$ we have a type $D(x, y, p)$, and

- for every $a : A$ we have an element $d(a) : D(a, a, \mathsf{refl}_a)$,

then

- there exists an element $\mathsf{ind}_{=_A}(D, d, x, y, p) : D(x, y, p)$ for *every* two elements $x, y : A$ and $p : x =_A y$, such that $\mathsf{ind}_{=_A}(D, d, a, a, \mathsf{refl}_a) \equiv d(a)$.

In other words, given dependent functions

$$D : \prod_{x,y:A} (x = y) \to \mathcal{U}$$
$$d : \prod_{a:A} D(a, a, \mathsf{refl}_a)$$

there is a dependent function

$$\mathsf{ind}_{=_A}(D, d) : \prod_{(x,y:A)} \prod_{(p:x=y)} D(x, y, p)$$

such that

$$\mathsf{ind}_{=_A}(D, d, a, a, \mathsf{refl}_a) \equiv d(a) \tag{0.0.1}$$

for every $a : A$.

Lemma 0.0.2. *For every type A and every $x, y : A$ there is a function*

$$(x = y) \to (y = x)$$

*denoted $p \mapsto p^{-1}$, such that $\mathsf{refl}_x{}^{-1} \equiv \mathsf{refl}_x$ for each $x : A$. We call p^{-1} the **inverse** of p.*

First proof. Assume given $A : \mathcal{U}$, and let $D : \prod_{(x,y:A)}(x = y) \to \mathcal{U}$ be the type family defined by $D(x, y, p) :\equiv (y = x)$. In other words, D is a function assigning to any $x, y : A$ and $p : x = y$ a type, namely the type $y = x$. Then we have an element

$$d :\equiv \lambda x.\, \mathsf{refl}_x : \prod_{x:A} D(x, x, \mathsf{refl}_x).$$

Thus, the induction principle for identity types gives us an element $\mathsf{ind}_{=_A}(D, d, x, y, p) : (y = x)$ for each $p : (x = y)$. We can now define the desired function $(-)^{-1}$ to be $\lambda p.\, \mathsf{ind}_{=_A}(D, d, x, y, p)$, i.e. we set $p^{-1} :\equiv \mathsf{ind}_{=_A}(D, d, x, y, p)$. The conversion rule (0.0.1) gives $\mathsf{refl}_x{}^{-1} \equiv \mathsf{refl}_x$, as required. □